

Top AI Video Clipping Tools

<https://gemini.google.com/app/efb535ba8fb54d09>

User prompt: WHAT ARE THE TOP 10 VIDEO CLIPPING TOOLS AND WHAT ARE THE DIFFERENT STYLES THAT ARE THE MOST MOST POPULAR AND BEING USED BY THESE TOOLS GIVE ME A TABLE COMPARISION PLEASE

Response: The top 10 AI video clipping tools are specialized for different workflows, ranging from automated podcast splitting to high-volume agency production.

1. **Opus Clip**: Recognized as the leading automated clipper, featuring AI virality scoring and "ClipAnything" dialogue and visual analysis to find high-retention moments.
2. **Munch**: Focuses heavily on extracting clips with high viral potential based on current social media trends, offering strong automatic subject tracking.
3. **Descript**: Uses a text-based editing approach, making it the top choice for podcasters who want to cleanly cut video by simply highlighting and deleting the transcript text.
4. **Vizard.ai**: Optimized for team workflows, business content, and webinars, offering strong collaboration features and shared workspaces.
5. **Ssemble**: Built for high-volume creators and agencies with per-video pricing rather than per-minute, offering full editing suites, stock B-roll, and API access.
6. **Submagic**: Specializes heavily in generating hyper-dynamic captions, auto-zooms, and automatic B-roll insertion for maximum viewer retention.
7. **Klap**: Designed to rapidly turn YouTube links into ready-to-publish TikToks, Reels, and Shorts with strong face-tracking and auto-reframing.
8. **Vidyo.ai**: Tailored for podcasters and interviewers, featuring highly customizable templates for dynamic split-screen speaker layouts.
9. **CapCut**: While primarily a manual mobile and desktop editor, its built-in AI auto-clipping, auto-captioning, and deep template library make it a staple for creators wanting fine-tuned control.
10. **Loopdesk**: Highly rated for directed, batch clipping, making it ideal for efficiently processing large volumes of podcast episodes at once.

Most Popular Editing Styles

AI tools are specifically trained to replicate high-retention formats popularized by top creators on TikTok, Instagram Reels, and YouTube Shorts:

- **The "Hormozi" Style (Dynamic Typography)**: Characterized by single-word or short-phrase captions flashing in the center of the screen. It uses highlighted keywords in distinct colors (often yellow or green), popping emojis, and aggressive jump-cuts to eliminate dead air.
- **Split-Screen / Podcast Layout**: Stacks two or more speakers vertically. The AI actively detects who is speaking and frames them, keeping the context of the interview clear without awkward horizontal panning.
- **Faceless / B-Roll Driven**: Uses continuous, highly relevant stock footage or AI-generated imagery that changes every 2 to 3 seconds over a voiceover. This keeps visual stimulation high for educational or storytelling content.
- **Gameplay / Reaction Split**: Places a primary speaker (often framed in a circle or top box) over engaging, unrelated background footage (like GTA V parkour, Minecraft, or satisfying kinetic sand) to artificially retain viewer attention during long monologues.
- **Minimalist / Corporate Clean**: Features subtle, standard lower-third captions with smooth transitions and minimal sound effects, preferred by B2B brands, consultants, and webinar clippers.

Tool Comparison

Tool	Primary Use Case	Standout Feature	Pricing Model
Opus Clip	Viral social media shorts	Virality Score & ClipAnything AI	Freemium (Per-minute)
Munch	Trend-based repurposing	Trend-analysis algorithm	Paid Subscription
Descript	Podcast & audio-first edits	Text-based timeline editing	Freemium (Per-hour transcription)
Vizard.ai	B2B & team collaboration	Shared workspaces & approvals	Paid Subscription
Ssemble	High-volume clipping	Per-video pricing & API access	Paid Subscription (Per-video)
Submagic	High-retention editing	Hyper-dynamic captions & B-roll	Paid Subscription
Klap	YouTube to TikTok pipeline	Fast, reliable auto-reframing	Paid Subscription
Vidyo.ai	Interview & podcast clips	Advanced split-screen templates	Freemium
CapCut	Hybrid (AI + Manual)	Unmatched template library	Freemium / Pro Tier

Tool	Primary Use Case	Standout Feature	Pricing Model
Loopdesk	Batch podcast processing	Directed high-volume clipping	Paid Subscription

Sources

- 1. [Best Video Clipping Tools 2025: Why OpusClip Leads](#)
- 2. [12 Best Clipping Software for Video Creators in 2026 \(AI & Manual\)](#)
- 3. [Best AI Video Tools for Short-Form Clips in 2026 - BlitzReels](#)
- 4. [Best AI Clipping Tools in 2026: Tested on Real Podcasts - Loopdesk](#)

User prompt: Now give me different category of videos to clip meanign reels only from already viral videos / viral creators and tell me how they should be best utilized as per youutbe standartds to maximise profit keep in mind youtube ad revenue algorithm

Response: Taking clips from already viral videos or well-known creators is one of the fastest ways to gain views, but on YouTube, **it is a legal and algorithmic minefield**. Simply re-uploading someone else's viral clip with basic captions will trigger YouTube's **Reused Content Policy**. This results in instant demonetization or a rejected YouTube Partner Program (YPP) application.

To maximize profit, you must understand **how YouTube's Ad Revenue algorithm works for Shorts** and apply the **"Transformative Standard"** so your channel gets approved for ad revenue and high-paying brand deals.

1. Top 5 Video Categories for Viral Clipping

1. Business, Personal Finance & Mindset Podcasts

- **Source Material:** *The Diary Of A CEO, Alex Hormozi, My First Million, All-In Podcast, Joe Rogan.*
- **Why It Works:** Finance and business have the highest advertiser CPMs on YouTube. Financial institutions, trading platforms, and SaaS products bid aggressively for this audience.
- **Estimated Shorts RPM:** **\$0.10 – \$0.45 per 1,000 views** (up to 10x higher than general entertainment).

2. Science, Health & Tech Explainers

- **Source Material:** *Andrew Huberman, Lex Fridman, MKBHD,* tech launch events, AI keynotes.
- **Why It Works:** Complex topics compressed into quick, actionable health or tech "hacks" get massive shareability and bookmarks.
- **Estimated Shorts RPM:** **\$0.07 – \$0.20 per 1,000 views.**

3. True Crime, Interrogations & Legal Cases

- **Source Material:** Police bodycam footage, court trials, interrogation tapes (*JCS-style*), legal podcasts.
- **Why It Works:** High psychological curiosity keeps retention rates above 90–100%. Viewers rarely swipe away from a tense moment.
- **Estimated Shorts RPM:** **\$0.04 – \$0.12 per 1,000 views.**

4. Gaming, Streamer Moments & Creator Drama

- **Source Material:** Twitch/Kick streams (*Kai Cenat, xQc, IShowSpeed*), esports tournaments, podcast debates.
- **Why It Works:** Extreme view potential (10M+ views rapidly) driven by younger audiences and passionate fanbases.
- **Estimated Shorts RPM:** **\$0.01 – \$0.05 per 1,000 views** (low revenue per view, but high total view volume).

5. Stand-up Comedy, Pop Culture & Hot Takes

- **Source Material:** *Flagrant, Kill Tony,* comedy specials, celebrity interviews.
- **Why It Works:** High comment section engagement. Arguments and debates in the comments signal to YouTube's algorithm that the video is highly engaging, boosting its feed distribution.
- **Estimated Shorts RPM:** **\$0.01 – \$0.05 per 1,000 views.**

2. YouTube Monetization Rules: How to Avoid "Reused Content" Bans

YouTube evaluates whether a human editor added **significant original value**. To monetize viral clips safely, use the **3-Layer Transformation Rule** on every upload:

[Original Viral Clip] + [Your Commentary / Voiceover] + [Custom Visual Layer] = Monetizable

1. **Audio Commentary (Voiceover/Anchor):** Start the Short with your own voice or an intro framing the context (e.g., *"Here is why Alex Hormozi's advice on real estate actually fails in 2026..."* or *"Notice what the detective does right here..."*).
2. **Visual Overlays & Editing:** Do not leave the source video unedited. Insert 2D/3D visual diagrams, stock motion graphics, side-by-side comparisons, or motion-tracked highlights to visually alter the canvas.
3. **Editorial Re-framing:** Turn passive clips into active breakdowns. Debunk, explain, or contextualize the clip instead of just trimming it.

3. Algorithmic Tactics to Maximise Profit

1. Optimize for "Viewed vs. Swiped Away" Ratio

YouTube's primary metric for Shorts distribution is the ratio of people who watch versus swipe away:

- **The 2-Second Rule:** Cut all intros, introductory greetings, or filler words (*"So basically..."*, *"Um..."*). Start directly at the visual or emotional peak.
- **Text On-Screen (Hook):** Add a high-contrast headline overlay in the first 3 seconds (e.g., *"The \$100M Mistake"* or *"Watch His Hands Right Here"*).

2. Aim for Over 100% Average Percentage Viewed (APV)

If your 30-second Short achieves a 110% average view duration (meaning viewers re-watch or loop it), YouTube will push it to millions of feeds.

- **Seamless Looping:** Design the final sentence of your video to grammatically flow directly back into your opening sentence.

3. Diversify Beyond Shorts Ad Revenue

Shorts ad revenue alone pays around **\$100 to \$400 per 10 million views** in lower-CPM niches. To build a high-profit business around clipping:

- **Pinned Comments:** Place affiliate links (books, software, tools mentioned in the clip) in the pinned comment.
- **Long-Form Funneling:** Use the official YouTube "Related Video" feature on your Short to route viral short-form traffic directly to your own monetized, 10-minute deep-dive videos (where CPMs range from \$5 to \$20+).
- **Brand Sponsorships:** Once you reach 50,000+ subscribers, offer sponsored integrations or affiliate promos directly inside your dynamic captions.

4. Category Comparison & Monetization Strategy

Category	Typical Audience	RPM / CPM Potential	Monetization Risk (Copyright/Reused)	Best Transformation Technique	Secondary Profit Strategy
Business & Finance	Adults (25–45), High Purchasing Power	Very High (\$0.10–\$0.45)	Low (Informational / Fair Use)	Animated charts, key stat callouts, case-study voiceover	Financial affiliate programs, SaaS tools, newsletters
Science & Tech	Tech-savvy, Students, Professionals	High (\$0.07–\$0.20)	Low (Educational framing)	B-roll diagrams, split-screen explanations, software demos	Tech affiliate links, software sponsorships
True Crime / Legal	Broad Demographic, High Watch Time	Medium (\$0.04–\$0.12)	Medium (Sensationalism checks)	Timeline graphics, evidence zoom-ins, legal breakdown commentary	Channel memberships, merch, podcast sponsorships
Gaming & Streamers	Teens & Young Adults (13–24)	Low (\$0.01–\$0.05)	High (Streamer copyright strikes)	Frame-by-frame analysis, "reaction" split-screen, meme edits	Discord communities, gaming gear affiliates
Comedy & Hot Takes	General Audience, High Comment Volume	Low (\$0.01–\$0.05)	High (Media network Content ID)	Dual-perspective breakdown, fact-checking, added context	Brand integrations, social media growth service

Sources

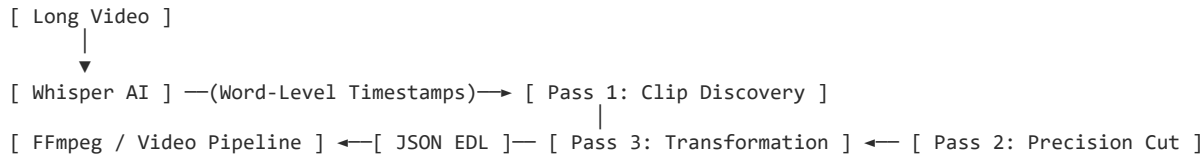
1. [YouTube Reused Content Policy: How to Stay Monetized in 2026](#)
2. [Most Profitable AI YouTube Shorts Niches in 2026 \(RPM Data\)](#)

User prompt: how should a PROGRAMM MAKING THIS BE INTRUCTED IF USING LLM TO DECIDE WHICH PART TO CUT AND HOW

Response: To build an automated video clipping tool powered by an LLM, the model must act as an **editor, curator, and scripting assistant**. Because LLMs cannot process raw video frames natively in real-time at scale, the program must feed the LLM **timecoded transcripts** (generated via tools like OpenAI Whisper, Deepgram, or AssemblyAI) and ask for a structured **Edit Decision List (EDL)** in JSON format.

A production-ready pipeline uses a **3-Pass Multi-Agent Prompting Strategy** to handle discovery, precision trimming, and YouTube monetization compliance.

Architecture Overview



Phase 1: Input Formatting (Word-Level Timestamps)

Feed the LLM transcript blocks that contain **exact word-level or sentence-level timestamps**. This gives the model the temporal context necessary to decide start and end points.

```

[00:01:23.100 -> 00:01:25.400] "If you want to build a business,"
[00:01:25.450 -> 00:01:27.800] "the first thing you need to kill"
[00:01:27.850 -> 00:01:29.200] "is your own ego."
  
```

Phase 2: The 3-Pass Prompting Strategy

Executing everything in a single prompt causes hallucinated timecodes and imprecise cuts. Breaking the process into three specialized passes yields significantly better results:

Pass 1: Clip Discovery & Virality Scoring

- **Objective:** Scan the long-form transcript to find candidate clips (30–60 seconds) that tell a complete, high-retention story.
- **LLM Instructions:**
 1. Identify segments containing a **strong hook** (curiosity gap, controversial statement, high emotion).
 2. Ensure the segment has a **payoff/core value** and a **clean resolution** (doesn't cut off mid-thought).
 3. Assign a virality/retention score (1–100) based on engagement criteria.

Pass 2: Precision Cut Refinement

- **Objective:** Tighten start and end boundaries down to the exact millisecond word-boundary.
- **LLM Instructions:**
 1. **Trim Filler Words:** Move the start timestamp *after* introductory fluff ("Um," "So," "Well basically").
 2. **Hard Hook Entry:** Ensure the clip's very first spoken sentence presents an immediate assertion or question.
 3. **Clean Outro Exit:** Stop the clip immediately after the punchline or key lesson, eliminating trailing off-topic banter.

Pass 3: Transformative Layer Generation (YouTube Monetization Compliance)

- **Objective:** Ensure the clip passes YouTube's **Reused Content Policy**.
- **LLM Instructions:**
 1. Generate a **5-second introductory script** for a custom AI voiceover host (e.g., "Here is why Alex Hormozi says ego kills early startups...").
 2. Identify 1–2 timestamps inside the clip where the video should pause or split for an **explainer overlay** or visual chart.
 3. Draft title options, dynamic caption highlights, and B-roll prompt keywords.

Phase 3: Production System Prompt Template

Below is a production-ready system prompt designed for **Pass 1 & Pass 2 combined**, enforcing a strict JSON output schema.

SYSTEM PROMPT: Video Segmentation Engine

You are an expert short-form video editor for TikTok, YouTube Shorts, and Instagram Reels. Your task is to analyze a timestamped video transcript and identify standalone viral clips optimized for maximum viewer

EVALUATION CRITERIA FOR CLIPS:

1. THE HOOK (First 3 Seconds): Must instantly create curiosity, shock, or high relevance.

2. **STANDALONE COHERENCE:** The clip must make complete sense without watching the rest of the original video.
3. **RETENTION DENSITY:** High ratio of valuable or entertaining information per second; zero filler.
4. **IDEAL DURATION:** Between 30 and 55 seconds.

RULES FOR TIMECODES:

- Start the clip at the exact word where the core idea or hook begins. Exclude intro filler ("So...", "You know...", "Li
- End the clip at the exact word where the thought finishes. Do not cut mid-sentence.
- Use the exact start_time and end_time provided in the input transcript format.

STRICT OUTPUT REQUIREMENTS:

Output ONLY a valid JSON object matching the requested schema. No conversational preamble, no markdown wrappers outside

Phase 4: Output JSON Schema (Edit Decision List)

Instructing the LLM to output structured JSON allows downstream video processing tools (such as Python scripts using **FFmpeg**, **MoviePy**, or **AutoEditor**) to automatically cut and render the final media.

Target JSON Structure

```
{
  "clips": [
    {
      "clip_id": 1,
      "title_hook": "The $100M Ego Mistake",
      "virality_score": 92,
      "reasoning": "High emotional punch in first 3 seconds with a clear business lesson.",
      "timestamps": {
        "start_time": "00:01:25.450",
        "end_time": "00:02:14.200"
      },
      "transformative_script": {
        "intro_voiceover": "Watch what happens when Alex Hormozi breaks down the number one killer of early-stage startu",
        "commentary_insertion_point": "00:01:45.000",
        "commentary_text": "Notice how he separates personal identity from business metrics here."
      },
      "b_roll_cues": [
        {
          "timestamp": "00:01:30.000",
          "prompt": "Cinematic shot of a failing business chart going down, dramatic lighting"
        }
      ],
      "dynamic_captions": {
        "highlight_keywords": ["kill", "ego", "$100M", "mistake"]
      }
    }
  ]
}
```

Critical Algorithmic Guardrails

To make your program resilient to real-world edge cases, implement these checks directly into your LLM prompt instructions:

1. **Context Window Boundary Checks:** When splitting a long video into chunks for the LLM, overlap transcript chunks by **2–3 minutes**. This prevents the LLM from missing clips that cross over chunk boundaries.
2. **Audio-Padding Offset:** Always program your video renderer (FFmpeg) to add a **−150ms padding** to the start and a **+200ms padding** to the end of the LLM's selected timestamps. This prevents accidental truncation of initial or final consonants.
3. **Pacing Rules:** Instruct the LLM to calculate **words-per-minute (WPM)**. Reject candidate clips with a WPM under 120, as slow speakers result in high swipe-away rates on short-form platforms unless accelerated in post-production ($1.15 \times -1.25 \times$).

User prompt: Okay, work on start work on editing clip editing.md something like that editing styles.md.md like make those with the different categories and the different styles that you gave. Let's work on that.

Response: Here is a production-ready editing_styles.md template designed for your workspace directory (OpenClaw or Antigravity) to guide both human editors and automated AI Agents on category classification, visual editing archetypes, and YouTube monetization compliance.

editing_styles.md - Video Categories, Editing Archetypes & Pipeline Rules

1. Source Categories & Content Niche Mapping

Category	Primary Source Content	Target Audience	RPM Potential	Optimal Duration	Monetization Strategy
Business & Finance	Podcasts (<i>Diary of a CEO</i> , <i>Alex Hormozi</i> , <i>My First Million</i>)	Adults (25–45), Entrepreneurs	Very High (\$0.10–\$0.45)	45–60 sec (60+ sec for TikTok)	SaaS affiliates, newsletter signups, high ad RPM
Science & Health	Academic/Health podcasts (<i>Huberman Lab</i> , <i>Lex Fridman</i>)	Biohackers, Tech workers	High (\$0.07–\$0.20)	50–75 sec	Supplement/health app affiliate links
Docu-Style & Deep Dives	Tech/Business drama (<i>SunnyV2</i> , <i>MagnatesMedia</i>)	General audience, Tech enthusiasts	Medium-High (\$0.08–\$0.18)	60–90 sec	Channel sponsorships, long-form video routing
True Crime & Legal	Interrogations, court trials, police bodycam	High-retention general viewers	Medium (\$0.04–\$0.12)	60–90 sec	Ad revenue (driven by >100% view time)
Gaming & Streamers	Twitch/Kick clips (<i>Kai Cenat</i> , <i>xQc</i> , esports)	Teens & Young Adults (13–24)	Low (\$0.01–\$0.05)	30–45 sec	Massive view volume, merch, Discord communities

2. Editing Archetypes & Visual Specifications

Style A: The "Hormozi" Kinetic Typography

- **Best Used For:** Business advice, productivity hacks, rapid educational content.
- **Text & Captions:**
 - 1–3 words per frame on-screen in center lower-third.
 - Bold sans-serif font (e.g., *Montserrat Black*, *Futura Bold*, *Impact*).
 - Highlight keywords dynamically in high-contrast yellow (#FFFF00) or lime green (#00FF00).
- **Visual & Pacing:**
 - Cut or hard zoom (1.1x–1.2x scale) every 1.5 to 2.5 seconds to eliminate dead air.
 - Insert popping emojis, sound effects (whoosh, pop), and motion graphics on key accent words.

Style B: Podcast & Split-Screen Framing

- **Best Used For:** Multi-speaker interviews, debates, and dynamic conversations (*The Diary Of A CEO*, podcast clips).
- **Aspect Ratio & Layout:**
 - 9:16 vertical canvas.
 - Dual horizontal split: Top box framed on Speaker A, bottom box framed on Speaker B.
 - Active speaker highlighting (border glow or dynamic camera shift to single frame when a speaker monologues).
- **Audio & Subtitles:**
 - Dynamic multi-color subtitles (Speaker A = Yellow, Speaker B = White).
 - Auto-framing with face tracking to keep heads centered.

Style C: Faceless B-Roll & Visual Explainer

- **Best Used For:** Science, health hacks, business case studies, and docu-style deep dives.
- **Visual Layering:**
 - Overlay stock motion graphics, Pexels/Shutterstock video footage, or AI images over the primary speaker voice.
 - B-roll swap frequency: Every 2 to 3 seconds.
 - Animated data graphics, line charts, or highlighted text callouts to visually prove spoken points.
- **Audio & Transitions:**
 - Subtle ambient background music ducked at -18dB to -22dB under speech.
 - Fast cross-dissolves or paper-rip transitions between B-roll cuts.

Style D: Gameplay / Split Reaction Overlay

- **Best Used For:** Long monologues, viral audio clips, and streamer highlights needing artificial retention boosts.
- **Layout:**
 - Top 60% of canvas: Primary speaker video or reaction clip.
 - Bottom 40% of canvas: Satisfying background footage (GTA V parkour, Subway Surfers, kinetic sand).
- **Pacing & Text:**
 - Clean center captions with dynamic tracking.
 - High-energy cuts to maintain background visual stimulation.

Style E: Minimalist Corporate Clean

- **Best Used For:** B2B content, tech tool showcases, executive highlights.
- **Style Guidelines:**
 - Subtle 2-line lower-third captions with clean typography (e.g., *Inter*, *Roboto*).
 - Minimal sound effects; soft ambient background track.
 - Focus on high video clarity, color grading, and crisp 1080p rendering.

3. YouTube Monetization & Originality Rules

To pass YouTube's **Reused Content Policy** and TikTok's **Originality Checks**, clips must apply the **3-Layer Transformation Standard**:

[Raw Source Clip] + [Original Audio/Commentary] + [Custom Visual Overlays] = Monetizable

1. **Audio Commentary Layer:** Every clip must open or integrate original audio (AI voiceover host or human commentary) providing context, debunking, or explaining the clip.
2. **Visual Overlay Layer:** Insert custom B-roll, on-screen text callouts, split-screen layouts, or side-by-side edits to ensure the original video frame is structurally modified.
3. **Pacing Optimization:** Trim all introductory fluff ("*Um*," "*So basically*"). Ensure the hook occurs within the first 2 seconds, and total duration exceeds 60 seconds if targeting TikTok Creator Rewards.

4. Edit Decision List (EDL) Schema for AI Agents

When the LLM processes transcripts to generate edit instructions, force the response to follow this target JSON structure:

```
{
  "clip_metadata": {
    "category": "Business & Finance",
    "editing_style": "Hormozi Kinetic Typography",
    "target_platform": "YouTube Shorts / TikTok",
    "estimated_rpm_category": "High"
  },
  "timestamps": {
    "start_time": "00:02:14.100",
    "end_time": "00:03:18.400",
    "duration_seconds": 64.3
  },
  "transformative_elements": {
    "intro_voiceover_script": "Here is why Alex Hormozi says high-volume sales outreach fails in 2026...",
    "b_roll_cues": [
      {
        "timestamp": "00:02:25.000",
        "type": "stock_video",
        "description": "Close up of person typing furiously on laptop with falling sales charts"
      }
    ],
    "text_overlays": [
      {
        "timestamp": "00:02:15.000",
        "text": "THE $100M SALES MISTAKE",
        "position": "top_center",
        "style": "bold_red_banner"
      }
    ]
  },
  "caption_settings": {
```

```

    "max_words_per_frame": 3,
    "font": "Montserrat Black",
    "primary_color": "#FFFFFF",
    "highlight_color": "#FFFF00"
  }
}

```

User prompt: Now, can this be sent to my anti-gravity IDE that's open in my laptop?

Response: I cannot directly push files onto your laptop's local hard drive from this web interface, but you can get `editing_styles.md` into your open Antigravity IDE immediately in one of two quick ways:

Method 1: Prompt your open Antigravity Agent (Recommended)

Because Antigravity features an integrated agent with direct access to your local workspace code editor and terminal, copy and paste this command directly into your open Antigravity chat window:

Prompt for Antigravity:

"Hey Antigravity, please create a new file named `editing_styles.md` in the workspace root. Paste our video category mappings, editing archetypes (Hormozi, Podcast, Faceless B-roll, Gameplay, Corporate), YouTube 3-layer transformation rules, and the target Edit Decision List (EDL) JSON schema into it so our clipping pipeline can reference it."

Once you send that prompt along with the markdown text, the Antigravity agent will autonomously open the editor, write the file, and save it to your local workspace directory.

Method 2: Save Directly to your Workspace Directory

If you prefer doing it manually inside your open IDE:

1. In your open Antigravity IDE, click **New File** in the left sidebar file tree.
2. Name the file `editing_styles.md`.
3. Paste the markdown content provided above into the editor and hit `Ctrl + S` (or `Cmd + S` on macOS).

Once saved in your project folder (or OpenClaw workspace path like `~/openclaw/workspace`), your trigger scripts and LLM prompts running inside Antigravity can read `editing_styles.md` to format JSON payloads for FFmpeg rendering.

User prompt: Okay, make different types of `editing.mds` for each style of video and make it so that we can talk over it and create over it, maybe different iterations, make it better over time. Keep that as a keep that in place, keep that architecture in place.

Response: Workspace Directory Architecture

To allow your workspace agent (Antigravity or OpenClaw) to dynamically load, discuss, and refine individual video style parameters over time without cluttering a single file, organize your workspace using a modular `/editing_styles/` directory:

```

~/openclaw/workspace/ (or Antigravity Root)
├── editing_styles/
│   ├── index.md                # Master Router & Selection Logic
│   ├── editing_hormozi.md      # Kinetic Typography (Business / Advice)
│   ├── editing_podcast.md      # Split-Screen (Interviews / Debates)
│   ├── editing_faceless.md     # B-Roll Explainer (Science / Docu-Style)
│   ├── editing_gameplay.md     # Split Reaction (Streams / Long Monologues)
│   └── editing_corporate.md    # Minimalist Clean (B2B / Webinars)

```

Each file includes an **Iteration & Feedback Log** at the top. This allows you to converse with Antigravity (e.g., *"Antigravity, update `editing_hormozi.md` to add a pop sound effect on every yellow word"*), and the agent will modify the parameters while preserving past version notes.

1. `editing_styles/index.md` (Master Router)

```
# Master Editing Router & Style Selection Matrix
```

```
## System Role
```

This file acts as the primary router for video editing pipelines. When processing a video transcript, evaluate the video

```
## Category-to-Style Mapping Rules
```

```
- IF `category` == "Business", "Finance", "Productivity", or "Single Speaker Advice":
  └─ Load `editing_styles/editing_hormozi.md`
- IF `category` == "Podcast", "Interview", "Debate", or "Multi-Speaker":
```

```

└─ Load `editing_styles/editing_podcast.md`
- IF `category` == "Science", "Tech Explainer", "Case Study", or "Docu-Style":
└─ Load `editing_styles/editing_faceless.md`
- IF `category` == "Streamer Highlight", "Gaming", or "Long Monologue without Visuals":
└─ Load `editing_styles/editing_gameplay.md`
- IF `category` == "B2B", "Webinar", "Corporate", or "SaaS Showcase":
└─ Load `editing_styles/editing_corporate.md`

## Pipeline Execution Flow
1. Run `trigger_engine.py` to identify source video URL.
2. Generate transcript with word-level timestamps via Whisper.
3. Match content niche to a category in this matrix.
4. Load the designated `editing_<style>.md` spec file.
5. Pass transcript + style spec parameters to LLM to output the final Edit Decision List (EDL) JSON.

```

2. editing_styles/editing_hormozi.md

Editing Spec: Hormozi Kinetic Typography

Iteration & Feedback Log

- **v1.0 (Initial):** Basic 1-3 word captions, yellow/green highlights, 2s jump cuts.
- **v1.1 (Target):** Add sound effect cues (whoosh, pop) on keyword highlights.
- **Agent Directive:** Update this section whenever parameters are tuned during agent discussions.

1. Style Definition

High-energy, fast-paced educational format designed for maximum retention on single-speaker advice[cite: 1].

2. Visual & Audio Rules

- **Aspect Ratio:** 9:16 (1080x1920)
- **Cut / Zoom Frequency:** Hard jump-cut or scale shift (1.0x to 1.15x) every 1.5 to 2.2 seconds to eliminate silence.
- **Captions:**
 - Max words per frame: 1-3 words centered in the lower-third.
 - Font: `Montserrat Black` or `Impact` (Uppercase, 48pt-56pt).
 - Primary Color: `#FFFFFF` (White) with 100% black outline (4px stroke).
 - Highlight Accent: `#FFFF00` (Yellow) or `#00FF00` (Lime Green) on high-value numbers or verbs.
- **Overlays:** Insert emojis or vector pop-ups on key accent words.
- **Audio:** SFX (`whoosh.wav`, `pop.wav`) synced to highlighted words.

3. Targeted EDL JSON Output Schema

```

```json
{
 "style": "hormozi_kinetic",
 "pacing": {
 "zoom_interval_seconds": 1.8,
 "cut_silence_threshold_ms": 150
 },
 "typography": {
 "font_family": "Montserrat Black",
 "font_size": 52,
 "highlight_color_hex": "#FFFF00",
 "stroke_color_hex": "#000000"
 },
 "sfx_cues": [
 {"timestamp": "00:00:03.200", "sound": "pop.wav"},
 {"timestamp": "00:00:08.500", "sound": "whoosh.wav"}
]
}

```

---

## 3. `editing\_styles/editing\_podcast.md`

```markdown

Editing Spec: Podcast & Split-Screen Framing

Iteration & Feedback Log

- **v1.0 (Initial):** Dual-stacked 9:16 frame with active speaker detection[cite: 1].
- **v1.1 (Target):** Auto-switch to full-screen 9:16 when a speaker monologues for >8 seconds.

1. Style Definition

Dual-speaker framing optimized for interviews, conversational podcasts, and debates (*Diary of a CEO*, *Huberman Lab*, *

2. Visual & Audio Rules

- **Aspect Ratio:** 9:16 (1080x1920)
- **Layout Architecture:**
 - **Top Box (0 to 50% height):** Face-tracked crop of Speaker A.
 - **Bottom Box (50% to 100% height):** Face-tracked crop of Speaker B.
- **Active Speaker Rule:**
 - If Speaker A talks continuously for >8 seconds, smoothly zoom Top Box to fill 100% canvas until Speaker B interjects
- **Captions:**
 - Position: Mid-screen divider line.
 - Font: `Inter Bold` or `Futura`.
 - Color Coding: Speaker A = `#FFFF00` (Yellow text), Speaker B = `#FFFFFF` (White text).

3. Targeted EDL JSON Output Schema

```
```json
{
 "style": "podcast_split_screen",
 "layout": {
 "type": "dual_stack_vertical",
 "speaker_a_crop_box": [0, 0, 1080, 960],
 "speaker_b_crop_box": [0, 960, 1080, 1920]
 },
 "speaker_colors": {
 "speaker_a_hex": "#FFFF00",
 "speaker_b_hex": "#FFFFFF"
 }
}
```

---

### ## 4. `editing\_styles/editing\_faceless.md`

```
```markdown
# Editing Spec: Faceless B-Roll & Visual Explainer
```

Iteration & Feedback Log

- **v1.0 (Initial):** Stock B-roll overlay every 2-3 seconds over transcript voiceover[cite: 1].
- **v1.1 (Target):** Add animated line charts and text stat callouts on financial/scientific figures.

1. Style Definition

Documentary and explainer style used for science, health, business case studies, and true crime breakdowns[cite: 1].

2. Visual & Audio Rules

- **Aspect Ratio:** 9:16 (1080x1920)
- **Visual Layering:**
 - Source video is suppressed or used sparingly (20% of duration).
 - High-density B-roll overlay (Pexels, stock video, AI images) changing every 2.0 to 3.0 seconds.
 - Motion: Continuous 1.05x subtle zoom (Ken Burns effect) on static images.
- **Captions & Graphics:**
 - Subtitles: Clean 2-line lower-third with soft dark backdrop box (50% opacity).
 - Stat Callouts: Animated center cards when numbers or stats are mentioned (e.g., "\$10,000/mo" or "78% Increase").
- **Audio:** Low background track ducked at -20dB underneath voiceover.

3. Targeted EDL JSON Output Schema

```
```json
{
 "style": "faceless_explainer",
 "b_roll_frequency_seconds": 2.5,
 "background_audio_duck_db": -20,
 "graphics_triggers": [
 {
 "timestamp": "00:00:12.100",
 "type": "stat_card",
 "main_text": "+140% Growth",
 "sub_text": "Q3 Revenue Surge"
 }
]
}
```

---

### ## 5. `editing\_styles/editing\_gameplay.md`

```
```markdown
# Editing Spec: Split Reaction / High Retention Overlay
```

```

## Iteration & Feedback Log
- **v1.0 (Initial):** Top 60% clip / Bottom 40% gameplay footage.
- **v1.1 (Target):** Auto-select background type based on mood (GTA V parkour for fast audio; kinetic sand for deep mono)

## 1. Style Definition
Designed for streamer clips, audio monologues, or viral discussions that lack high visual movement.

## 2. Visual & Audio Rules
- **Aspect Ratio:** 9:16 (1080x1920)
- **Canvas Layout:**
  - **Top 60% (1080x1152):** Primary clip / reaction camera centered on speaker.
  - **Bottom 40% (1080x768):** High-retention stock video (GTA V parkour, Subway Surfers, satisfying craft loops).
- **Audio Rules:**
  - Mute audio on bottom footage completely.
  - Primary voiceover normalized to -1dB peak.
- **Captions:** Center-aligned kinetic captions placed over the boundary line between top and bottom clips.

## 3. Targeted EDL JSON Output Schema
```json
{
 "style": "gameplay_split_overlay",
 "layout_split_ratio": "60_40",
 "background_media_type": "gta5_parkour",
 "top_frame_crop": [0, 0, 1080, 1152],
 "bottom_frame_crop": [0, 1152, 1080, 1920]
}

6. `editing_styles/editing_corporate.md`
```markdown
# Editing Spec: Minimalist Corporate Clean

## Iteration & Feedback Log
- **v1.0 (Initial):** Clean lower-thirds, subtle transitions, crisp 1080p output[cite: 1].
- **v1.1 (Target):** Add custom brand color hex variable to JSON output.

## 1. Style Definition
Refined format tailored for B2B executives, SaaS software demos, tech webinars, and corporate announcements[cite: 1].

## 2. Visual & Audio Rules
- **Aspect Ratio:** 9:16 (1080x1920) or 1:1 (1080x1080) for LinkedIn/X.
- **Pacing:** Standard natural cadence. Zero aggressive jump cuts.
- **Captions:**
  - Style: Elegant, minimal lower-third banner.
  - Font: `Inter` or `Roboto Medium`.
  - Color: Dark gray/navy text backdrop (`#1A1D20`) with white text.
- **Graphics:** Subtle side-slide corporate transitions and polished lower-third name tags.

## 3. Targeted EDL JSON Output Schema
```json
{
 "style": "corporate_clean",
 "typography": {
 "font_family": "Inter",
 "font_size": 36,
 "background_box_color": "#1A1D20",
 "text_color": "#FFFFFF"
 },
 "transitions": "fade_cross_dissolve_200ms"
}

```

## Direct Prompt to Automate Creation in Antigravity

Copy and paste this exact command into your open Antigravity chat window to generate the entire architecture automatically:

### Antigravity Execution Prompt:

"Hey Antigravity, create a new directory named `editing_styles` in our workspace root. Inside `/editing_styles/`, generate the following files: `index.md`, `editing_hormozi.md`, `editing_podcast.md`, `editing_faceless.md`, `editing_gameplay.md`, and `editing_corporate.md` using the exact structure, category rules, iteration logs, and EDL JSON schemas outlined above. Once created,

update our main processing pipeline so that before calculating timestamps, the LLM reads `/editing_styles/index.md` to load the appropriate style parameters."

User prompt: Okay, now how do I make this like how do I feed this editing style the MD into my LLM or like do I even need to push it to LLM can this just be done like be taken care of like programmatically will the editing styles make a difference in the program itself and the editing itself or will it just be done by the AI how would you suggest I do it

Response: The most effective setup uses a **Hybrid Architecture**: the **LLM acts as the Creative Director** (deciding *where* to cut and *what* text/overlays to generate), while your **Python/FFmpeg program acts as the Video Editor** (deterministically handling layout math, cropping, font rendering, and file assembly).

You do **not** feed the entire `editing_styles` folder to the LLM at once. Instead, your Python pipeline dynamically loads the specific `editing_<style>.md` file into the LLM's system prompt to get a structured JSON payload, which Python then executes programmatically.

### 1. How to Feed editing\_styles.md to the LLM

In your Python automation script (e.g., inside Antigravity or OpenClaw), read the relevant markdown file as a string and inject it directly into the LLM's system context when requesting clip selection.

```
import json
import openai

def load_style_spec(style_name: str) -> str:
 """Dynamically loads the style rules from your workspace markdown directory."""
 filepath = f"editing_styles/editing_{style_name}.md"
 with open(filepath, "r", encoding="utf-8") as f:
 return f.read()

def get_clip_edl(transcript_text: str, style_name: str = "hormozi"):
 style_instructions = load_style_spec(style_name)

 system_prompt = f"""
 You are an expert viral video editor. Analyze the transcript and extract high-retention clips.

 STRICT STYLE GUIDELINES TO FOLLOW:
 {style_instructions}

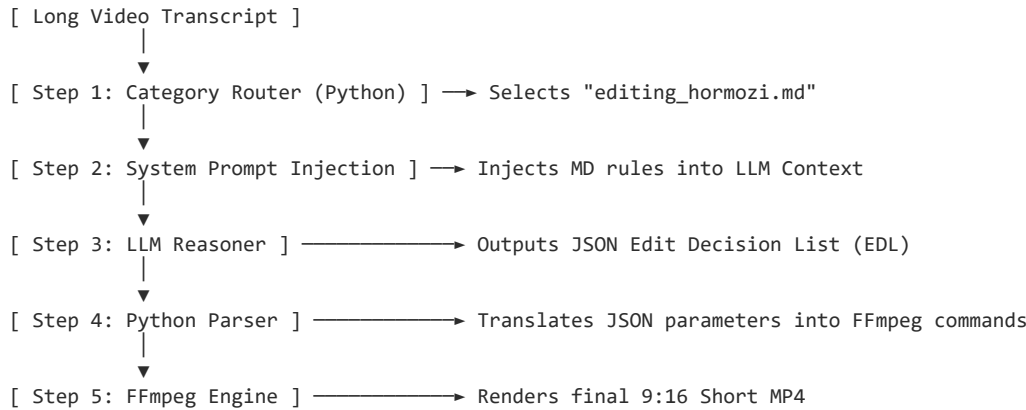
 Output ONLY a valid JSON Edit Decision List (EDL) following the schema defined in the style instructions.
 """

 response = openai.chat.completions.create(
 model="gpt-4o", # or local Ollama / vLLM endpoint
 response_format={"type": "json_object"},
 messages=[
 {"role": "system", "content": system_prompt},
 {"role": "user", "content": f"Transcript:\n{transcript_text}"}
]
)
 return json.loads(response.choices[0].message.content)
```

### 2. Division of Labor: AI vs. Programmatic Execution

Task	Handled By	Why?
Clip Discovery & Hooks	LLM (AI)	Requires semantic understanding of storytelling, humor, curiosity gaps, and topic pacing.
Transformative Commentary / Captions	LLM (AI)	Writes the 5-second voiceover scripts, selects key phrases to highlight, and picks B-roll keywords.
Video Cropping & Auto-Framing	Program (Python/OpenCV)	Face-tracking algorithms (e.g., MediaPipe or OpenCV) calculate pixel coordinates for 9:16 framing faster and more accurately than an LLM.
Subtitle Rendering & Font Styling	Program (FFmpeg/MoviePy)	FFmpeg applies the exact font family, hex color (#FFFF00), stroke width, and position specified in the style preset.
Audio Ducking & SFX Alignment	Program (FFmpeg)	Merges audio tracks, ducks background music to -20dB, and aligns .wav sound effects to exact milliseconds.

### 3. Recommended Pipeline Architecture



## 4. Why This Architecture Makes Iteration Easy

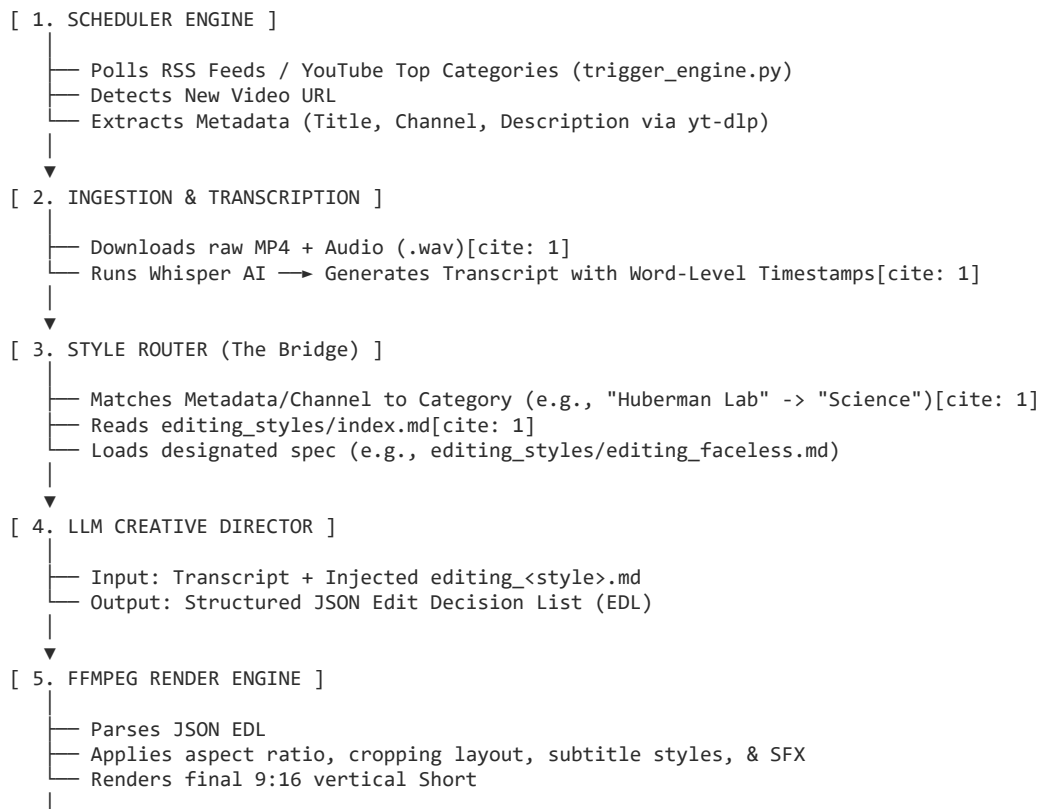
- Decoupled Code & Style:** You never have to edit python code (render.py or ffmpeg\_builder.py) when you want to change font size, subtitle color, or pacing. You simply edit editing\_hormozi.md in your text editor.
- Deterministic Output:** Keeping rendering inside Python/FFmpeg prevents the LLM from making hallucinated visual mistakes (like setting an aspect ratio to 90:16 or picking a non-existent font).
- Conversational Refinement in Antigravity:** You can tell your Antigravity agent: "Antigravity, open editing\_podcast.md and change the subtitle font size from 36 to 44 and make the Speaker A highlight color cyan." The agent updates the .md file, and the next run automatically applies the changes.

User prompt: Okay, by the way, my architecture works is uh it's it runs a scheduler, gets the top category like top videos and then it uh goes and edits edits it according to my style. So, how would these this architecture fit into that?

Response: Your editing\_styles/ directory fits seamlessly into your scheduler pipeline by acting as a **dynamic parameter engine** between the ingestion phase and the rendering engine.

Here is the complete end-to-end flow showing how your scheduler fetches top videos, determines their style, generates the EDL via the LLM, and passes the parameters to FFmpeg.

## The End-to-End Architecture Flow



```

▼
[6. STATE LOGGING & DISTRIBUTION]
├─ Saves video ID to seen_videos.json (Prevents duplicate processing)
└─ Places final MP4 in /output/ folder for scheduling/uploading[cite: 1]

```

## How to Integrate Style Routing into trigger\_engine.py

Update your main processing function inside your Python scheduler script so that it automatically selects and injects the proper style rules:

```

import json
import os
import openai
from yt_dlp import YoutubeDL

1. Channel-to-Style Mapping Lookup
CHANNEL_STYLE_MAP = {
 "Huberman Lab": "faceless",
 "Diary of a CEO": "podcast",
 "Alex Hormozi": "hormozi",
 "Kai Cenat": "gameplay"
}

def get_style_for_video(channel_name: str, video_title: str) -> str:
 """Returns the style name based on target channel or title keywords."""
 # Check exact channel match first
 if channel_name in CHANNEL_STYLE_MAP:
 return CHANNEL_STYLE_MAP[channel_name]

 # Fallback keyword checks
 title_lower = video_title.lower()
 if any(k in title_lower for k in ["interview", "podcast", "vs"]):
 return "podcast"
 elif any(k in title_lower for k in ["business", "money", "sales"]):
 return "hormozi"

 # Default fallback style
 return "faceless"

def load_style_spec(style_name: str) -> str:
 """Reads the corresponding markdown spec from editing_styles/"""
 spec_path = f"editing_styles/editing_{style_name}.md"
 if os.path.exists(spec_path):
 with open(spec_path, "r", encoding="utf-8") as f:
 return f.read()
 # Fallback to index if specific file missing
 with open("editing_styles/index.md", "r", encoding="utf-8") as f:
 return f.read()

def process_video_pipeline(video_url: str, channel_name: str, video_title: str):
 """Main pipeline execution triggered by scheduler."""
 print(f"📺 Processing video: {video_title} from {channel_name}")

 # Step A: Ingestion & Transcription (Whisper)
 # audio_file = download_audio(video_url)
 # transcript_data = transcribe_with_whisper(audio_file)
 transcript_data = "..." # Word-level timestamps from Whisper

 # Step B: Determine Style and Load Markdown Spec
 style_name = get_style_for_video(channel_name, video_title)
 style_instructions = load_style_spec(style_name)
 print(f"🎨 Matched Editing Style: editing_{style_name}.md")

 # Step C: Prompt LLM with Injected Style Rules
 system_prompt = f"""
 You are an expert short-form video editor. Select the top viral moments from the transcript.

 CRITICAL STYLE SPECIFICATIONS TO FOLLOW:
 {style_instructions}

 Output ONLY a valid JSON EDL following the schema in the spec instructions.
 """

```

```
response = openai.chat.completions.create(
 model="gpt-4o",
 response_format={"type": "json_object"},
 messages=[
 {"role": "system", "content": system_prompt},
 {"role": "user", "content": f"Transcript:\n{transcript_data}"}
]
)

edl_json = json.loads(response.choices[0].message.content)

Step D: Pass JSON EDL to FFmpeg Render Engine
render_final_clip(video_url, edl_json)
print("✅ Render complete! Clip saved to output directory.")
```

## Why This Architecture Works Best

1. **Zero Hardcoded Logic:** If you want to change subtitle colors for Hormozi clips or adjust the split-screen ratio for podcasts, you edit `editing_hormozi.md` or `editing_podcast.md`. You **never** need to touch `trigger_engine.py` or stop your running scheduler.
  2. **Infinite Category Scaling:** When you want to add a new video category (e.g., `editing_docu.md`), you simply create the markdown file in `/editing_styles/` and add one line to `CHANNEL_STYLE_MAP`.
  3. **Resilient & Modular:** If the LLM produces a clip, Python parses the exact JSON format defined by that style's markdown file and constructs the matching FFmpeg filters dynamically.
-